

AWS Architecture — Complete Request Flow

If you don't know **AWS Architecture**, consider the interview gone!

Let's say you have a website and a user sends a request.

The first thing we need is **DNS**.

1. Route 53 — DNS

The request first goes through **Route 53**, AWS's DNS service.

DNS basically converts a human-readable domain like:

`www.myapp.com`

into the appropriate destination, such as an IP address or an AWS resource.

So Route 53 helps the user find **where our application is hosted**.

2. CloudFront — CDN

Next, if we're using **CloudFront**, the request can go through CloudFront.

CloudFront is AWS's **Content Delivery Network**, or CDN.

It has edge locations around the world and can cache frequently accessed content closer to users.

For example:

- JavaScript files
- CSS
- Images
- Videos
- Other static content

This reduces latency and makes content load faster.

3. S3 — Object Storage

Now where do we actually store those frontend files?

We can use **Amazon S3**.

S3 is AWS's **object storage service**.

We can store things like:

- React/Next.js build files
- Images
- Videos
- PDFs
- Documents
- Backups

So our frontend flow can look like:

Route 53 → CloudFront → S3 → Frontend

4. API Gateway — API Entry Point

Now suppose our frontend wants some data from the backend.

For example:

`GET /users/123`

The request can go through **API Gateway**.

API Gateway acts as an **entry point for our APIs**.

It can handle things like:

- Routing
- Authentication
- Authorization
- Throttling
- Rate limiting
- Access control

So instead of exposing every backend service directly to the internet, API Gateway can act as a controlled entry point.

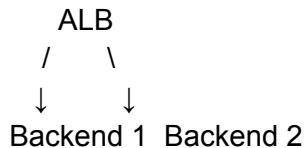
5. Load Balancer — Distributing Traffic

Now our request needs to reach the backend.

In a production system, we usually don't want a single backend server handling everything.

We can use an **Application Load Balancer**, or ALB.

For example:



The load balancer distributes incoming traffic across multiple backend instances or containers.

It can also perform **health checks** and stop sending traffic to unhealthy instances.

6. Backend — EC2, ECS or Lambda

Now we need to actually run our backend application.

Suppose we're using **Java + Spring Boot**.

AWS gives us several ways to run it.

The three important ones are:

EC2, ECS and Lambda.

7. EC2 — Virtual Server

EC2 is basically a virtual server in AWS.

You can choose:

- CPU
- RAM
- Storage
- Operating system

Then you install Java, deploy your Spring Boot application and run it.

The advantage is **more control**.

But you also have more responsibility:

- Server configuration
- OS updates
- Scaling
- Security
- Runtime management

Think of EC2 as:

"AWS, give me a server."

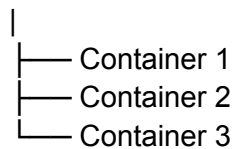
8. ECS — Container Orchestration

Now suppose our Spring Boot application is packaged as a **Docker container**.

We can use **ECS — Elastic Container Service** to run and manage those containers.

For example:

ECS



ECS can run containers using either **EC2 infrastructure or Fargate**.

With ECS + EC2, we manage the underlying EC2 capacity.

With **ECS + Fargate**, AWS manages the underlying compute infrastructure for us.

Think of ECS as:

"AWS, manage and run my containers."

9. Lambda — Serverless Compute

The third option is **Lambda**.

Lambda is a **serverless compute service**.

You don't manage the underlying servers.

You simply deploy your function, and AWS runs it when it's triggered by an event or request.

It's particularly useful for:

- Event-driven workloads
- Lightweight processing
- APIs
- Automation
- Background tasks

Think of Lambda as:

"AWS, just run this function when I need it."

10. SQS — Message Queue

Now let's say the user places an order.

The order is successfully created, but we also need to:

- Send an email
- Generate an invoice
- Send a notification

We don't want the user to wait for all these tasks.

So we can use **SQS — Simple Queue Service**.

The backend puts a message into the queue:

"Send order confirmation email"

The user immediately gets:

"Order placed successfully."

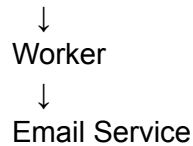
Meanwhile, a background worker picks up the message from SQS and processes it.

So:

Backend



SQS



SQS helps us build **asynchronous and decoupled systems**.

11. RDS — Relational Database

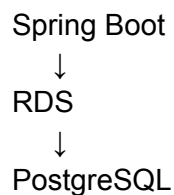
Obviously, our application needs a database.

If we need a **relational database**, we can use **RDS — Relational Database Service**.

For example:

- PostgreSQL
- MySQL
- MariaDB
- SQL Server
- Oracle

Our Spring Boot application can communicate with:



AWS manages many of the database administration tasks for us.

12. DynamoDB — NoSQL Database

If our application needs a **NoSQL database**, we can use DynamoDB.

DynamoDB is AWS's managed NoSQL database and supports key-value and document-style data models.

For example:

User ID → User Data

It's designed for applications that need highly scalable and low-latency data access.

So:

RDS → Relational

DynamoDB → NoSQL

13. S3 — File Storage

We've already seen S3 for frontend files.

But S3 can also be used for application data such as:

- User profile images
- Videos
- PDFs
- Documents
- Backups

For example:

User
↓
Spring Boot
↓
S3
↓
profile-picture.jpg

For large files, we generally don't want to store them directly inside our database.

14. IAM — Security & Permissions

Now comes security.

We need to answer:

"Who can access what?"

That's where **IAM — Identity and Access Management** comes in.

For example, suppose our Spring Boot application needs to read files from S3.

Instead of hardcoding AWS credentials inside the application, we can use an appropriate **IAM role and permissions**.

So think of IAM as:

"Who is allowed to do what?"

15. CloudWatch — Monitoring

Finally, we have to monitor the entire system.

That's where **CloudWatch** comes in.

It can help us monitor:

- Logs
- Metrics
- Errors
- Application health
- Alarms

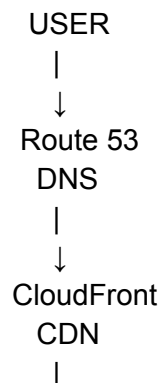
For example, if our backend suddenly starts returning too many errors, CloudWatch can help us detect the problem and trigger an alarm.

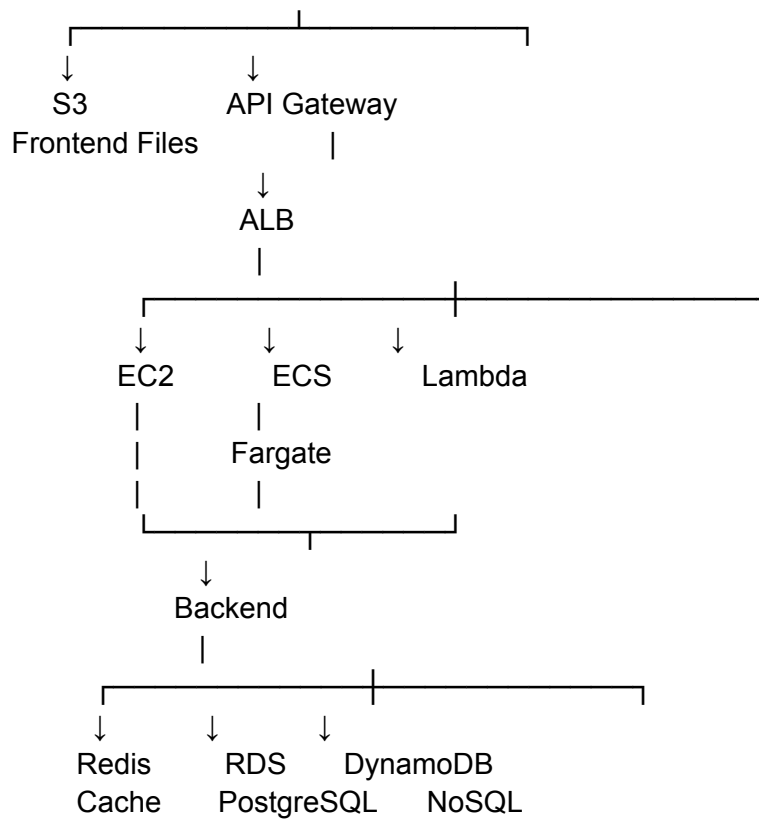
So:

CloudWatch = Monitoring + Logs + Metrics + Alarms

Putting Everything Together

So our simplified AWS architecture looks like this:





Backend



SQS



Background Worker



Email / Notifications

S3 → Images / Videos / Files

IAM → Permissions

CloudWatch → Monitoring

VPC → Networking

The easiest way to remember it:

Route 53 → Find the application

CloudFront → Deliver content faster

S3 → Store files

API Gateway → API entry point

ALB → Distribute traffic

EC2 → Virtual server

ECS → Run/manage containers

Lambda → Run functions without managing servers

SQS → Asynchronous message queue

RDS → Relational database

DynamoDB → NoSQL database

IAM → Permissions

CloudWatch → Monitoring

VPC → Private AWS network

Once you understand **why each component exists and how a request flows through them**, AWS architecture becomes much easier to explain in an interview.